

Raw CSVs to Parquet

The 8 Kaggle tables (about 847 MB of CSV) converted with DuckDB, then counted.

In [3]

```
con = duckdb.connect()
for f in sorted(RAW.glob('*.*csv')):
    if not (out := PQ/f'{f.stem}.parquet').exists(): con.execute(f"COPY (SELECT *
FROM read_csv_auto('{f}')) TO '{out}' (FORMAT PARQUET)"); print('done', f.stem)
```

In [4]

```
t = lambda n: f"'{PQ}/{n}.parquet'"
pd.DataFrame([(f.stem, con.execute(f"SELECT COUNT(*) FROM {t(f.stem)}").fetchone()
[0], len(con.execute(f"SELECT * FROM {t(f.stem)} LIMIT 0").description),
round(f.stat().st_size/1e6, 1)) for f in sorted(PQ.glob('*.*parquet'))], columns=
['table', 'rows', 'cols', 'parquet_MB'])
```

Out

table	rows	cols	parquet_MB
campaign_desc	30	4	0.0
campaign_table	7208	3	0.0
causal_data	36786524	5	46.5
coupon	124548	3	0.7
coupon_redempt	2318	4	0.0
hh_demographic	801	8	0.0
product	92353	7	1.1
transaction_data	2595732	12	24.0

Do all the keys join?

Every foreign key checked with NOT EXISTS, which never multiplies rows when a parent key repeats.

In [8]

```
q("""
WITH hh AS (SELECT DISTINCT household_key FROM transaction_data)
SELECT 'transaction_data -> product' AS link, COUNT(*) AS n,
      COUNT(*) FILTER (WHERE NOT EXISTS (SELECT 1 FROM product p WHERE
p.PRODUCT_ID = x.PRODUCT_ID)) AS orphans
FROM transaction_data x
UNION ALL
SELECT 'hh_demographic -> transaction_data', COUNT(*),
      COUNT(*) FILTER (WHERE NOT EXISTS (SELECT 1 FROM hh WHERE hh.household_key =
x.household_key))
FROM hh_demographic x
...
SELECT 'causal_data -> product', COUNT(*),
      COUNT(*) FILTER (WHERE NOT EXISTS (SELECT 1 FROM product p WHERE
p.PRODUCT_ID = x.PRODUCT_ID))
FROM causal_data x
""").assign(orphan_pct=lambda d: (100 * d.orphans / d.n).round(2))
```

Out

link	n	orphans	orphan_pct
transaction_data -> product	2595732	0	0.0
hh_demographic -> transaction_data	801	0	0.0
campaign_table -> campaign_desc	7208	0	0.0
campaign_table -> transaction_data	7208	0	0.0
coupon -> product	124548	0	0.0
coupon_redempt -> coupon	2318	0	0.0
coupon_redempt -> campaign_table	2318	0	0.0
causal_data -> product	36786524	0	0.0

Finding the data quirks

Negative or zero lines, and discounts stored with the wrong sign.

```
In [11]
q("""
SELECT COUNT(*) AS n_rows,
       COUNT(*) FILTER (WHERE SALES_VALUE < 0) AS negative_sales,
       COUNT(*) FILTER (WHERE SALES_VALUE = 0) AS zero_sales,
       COUNT(*) FILTER (WHERE QUANTITY < 0) AS negative_qty,
       COUNT(*) FILTER (WHERE QUANTITY = 0) AS zero_qty,
       COUNT(*) FILTER (WHERE RETAIL_DISC > 0) AS positive_retail_disc,
       COUNT(*) FILTER (WHERE COUPON_DISC > 0) AS positive_coupon_disc,
       COUNT(*) FILTER (WHERE COUPON_MATCH_DISC > 0) AS positive_match_disc,
       COUNT(*) FILTER (WHERE COUPON_DISC < 0) AS rows_with_coupon
FROM transaction_data
""")
```

Out

n_rows	2595732
negative_sales	0
zero_sales	18850
negative_qty	0
zero_qty	14466
positive_retail_disc	36
positive_coupon_disc	0
positive_match_disc	0
rows_with_coupon	36422

Campaign overlap per household

A self-join finds campaigns whose date windows overlap for the same household.

In [22]

```
q("""
WITH hc AS (
    SELECT c.household_key, d.CAMPAIGN, d.START_DAY, d.END_DAY
    FROM campaign_table c
    JOIN campaign_desc d ON d.CAMPAIGN = c.CAMPAIGN
),
per_hh AS (
    SELECT a.household_key, COUNT(DISTINCT a.CAMPAIGN) AS campaigns,
           COUNT(b.CAMPAIGN) AS overlapping_pairs
    FROM hc a
    LEFT JOIN hc b ON b.household_key = a.household_key AND b.CAMPAIGN > a.CAMPAIGN
                  AND a.START_DAY <= b.END_DAY AND b.START_DAY <= a.END_DAY

    GROUP BY 1
)
SELECT COUNT(*) AS targeted_households,
       MEDIAN(campaigns) AS median_campaigns,
       MAX(campaigns) AS max_campaigns,
       COUNT(*) FILTER (WHERE campaigns = 1) AS single_campaign,
       COUNT(*) FILTER (WHERE overlapping_pairs > 0) AS with_overlap,
       ROUND(100 * COUNT(*) FILTER (WHERE overlapping_pairs > 0) / COUNT(*), 1) AS
with_overlap_pct
FROM per_hh
""")
```

Out

targeted_households	1584
median_campaigns	4.0
max_campaigns	17
single_campaign	268
with_overlap	884
with_overlap_pct	55.8

Duplicates and reconciliation

Duplicated promotion flags found, then cleaned totals checked against the raw table.

```
In [7]
q("""
SELECT COUNT(*) AS duplicated_keys,
      COUNT(*) FILTER (WHERE n_display > 1 OR n_mailer > 1) AS conflicting_keys
FROM (
  SELECT PRODUCT_ID, STORE_ID, WEEK_NO, COUNT(DISTINCT display) AS n_display,
  COUNT(DISTINCT mailer) AS n_mailer
  FROM causal_data
  GROUP BY 1, 2, 3
  HAVING COUNT(*) > 1
)
""")
```

Out

	duplicated_keys	conflicting_keys
	15245	15245

```
In [25]
q(f"""
SELECT (SELECT ROUND(SUM(SALES_VALUE), 2) FROM transaction_data) AS raw_sales,
      (SELECT ROUND(SUM(sales_value), 2) FROM '{OUT}/fact_sales.parquet') AS
processed_sales,
      (SELECT ROUND(SUM(sales_value), 2) FROM '{OUT}/fact_sales.parquet' WHERE NOT
is_nonmerch AND NOT is_zero_line) AS merchandise_sales
""")
```

Out

raw_sales	8057463.08
processed_sales	8057463.08
merchandise_sales	7376290.62

One filtered view for every question

Audit rules applied once: merchandise only, no zero lines, stable weeks 16 to 101.

In [2]

```
con.execute("""
CREATE OR REPLACE VIEW sales AS
SELECT f.*, p.department, p.commodity, p.brand
FROM fact_sales f
JOIN dim_product p ON p.product_id = f.product_id
WHERE NOT f.is_nonmerch AND NOT f.is_zero_line AND f.week_no BETWEEN 16 AND 101
""");
```

In [3]

```
q("""
SELECT COUNT(DISTINCT household_key) AS households,
       COUNT(DISTINCT basket_id) AS baskets,
       ROUND(SUM(sales_value)) AS sales,
       ROUND(SUM(sales_value) / COUNT(DISTINCT basket_id), 2) AS avg_basket_value,
       ROUND(1.0 * COUNT(*) / COUNT(DISTINCT basket_id), 1) AS
avg_lines_per_basket,
       ROUND(1.0 * COUNT(DISTINCT basket_id) / COUNT(DISTINCT household_key) /
COUNT(DISTINCT week_no), 2) AS baskets_per_household_per_week,
       ROUND(SUM(sales_value) / COUNT(DISTINCT household_key) / COUNT(DISTINCT
week_no), 2) AS spend_per_household_per_week
FROM sales
""")
```

Out

households	2493
baskets	230892
sales	6814347.0
avg_basket_value	29.51
avg_lines_per_basket	10.2
baskets_per_household_per_week	1.08
spend_per_household_per_week	31.78

Which departments grew?

A fixed panel of households present at both ends, so panel enrolment does not look like growth.

```
In [4]
q("""
WITH panel AS (
    SELECT household_key
    FROM sales
    GROUP BY household_key
    HAVING MIN(week_no) <= 19 AND MAX(week_no) >= 98
),
d AS (
    SELECT s.department,
           SUM(CASE WHEN s.week_no <= 58 THEN s.sales_value ELSE 0 END) AS h1_sales,
           SUM(CASE WHEN s.week_no >= 59 THEN s.sales_value ELSE 0 END) AS h2_sales
    FROM sales s
    JOIN panel p ON p.household_key = s.household_key
    GROUP BY s.department
),
t AS (SELECT SUM(h2_sales) / SUM(h1_sales) - 1 AS store_growth FROM d)
SELECT department, ROUND(h1_sales) AS h1_sales, ROUND(h2_sales) AS h2_sales,
       ROUND(100 * (h2_sales / h1_sales - 1), 1) AS growth_pct,
       ROUND(100 * (h2_sales / h1_sales - 1 - store_growth), 1) AS vs_store_pts,
       (SELECT COUNT(*) FROM panel) AS panel_households
FROM d CROSS JOIN t
WHERE h1_sales >= 10000
ORDER BY vs_store_pts DESC
""")
```

Out

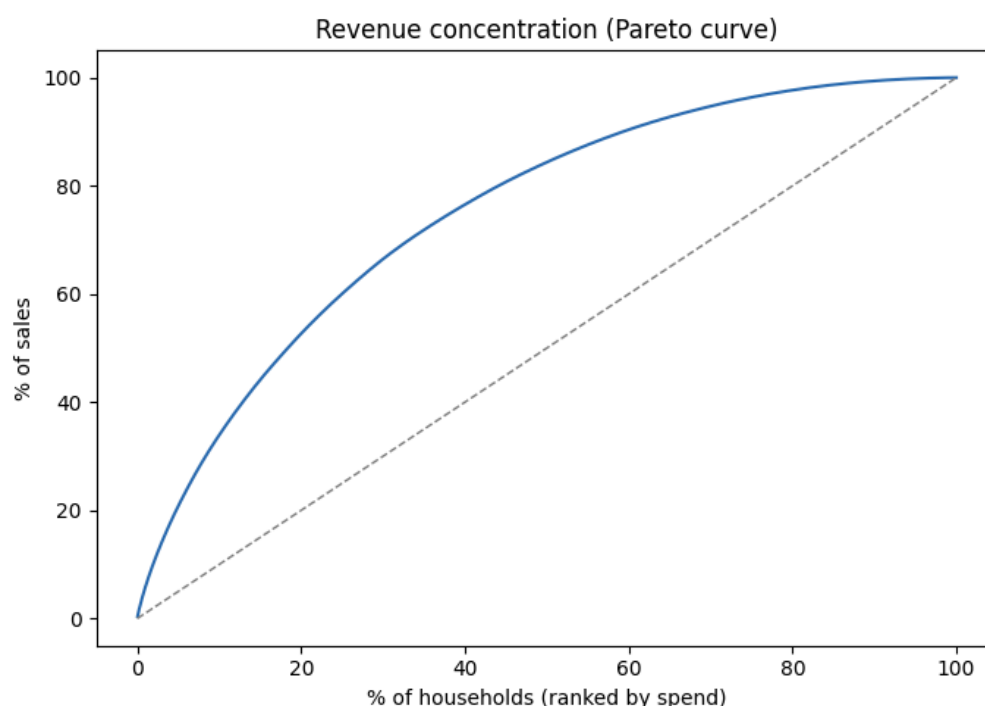
department	h1_sales	h2_sales	growth_pct	vs_store_pts
COSMETICS	11922.0	13871.0	16.3	10.6
SEAFOOD	10883.0	12541.0	15.2	9.5
SALAD BAR	10850.0	12425.0	14.5	8.8
NUTRITION	38211.0	43405.0	13.6	7.9
DELI	98152.0	108845.0	10.9	5.2
...				
SEAFOOD-PCKGD	24525.0	24348.0	-0.7	-6.4
FLORAL	16121.0	15954.0	-1.0	-6.8

How concentrated is revenue?

Window functions rank households and build the running share of sales in one query.

```
In [5]
pareto = q("""
WITH hh AS (
    SELECT household_key, SUM(sales_value) AS spend
    FROM sales
    GROUP BY household_key
)
SELECT household_key, spend,
    1.0 * ROW_NUMBER() OVER (ORDER BY spend DESC) / COUNT(*) OVER () AS
hh_share,
    SUM(spend) OVER (ORDER BY spend DESC ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) / SUM(spend) OVER () AS sales_share
FROM hh
ORDER BY spend DESC
""")
print(*[f"Top {int(p * 100)}% of households: {pareto.sales_share[pareto.hh_share <=
p].max():.1%} of sales" for p in (0.1, 0.2, 0.5)], sep='\n')
...
```

```
Top 10% of households: 33.9% of sales
Top 20% of households: 52.6% of sales
Top 50% of households: 84.4% of sales
```



RFM scores with CUME_DIST

Tied households get the same score, unlike NTILE. Simple rules turn scores into segments.

```
In [4]
con.execute("""
CREATE OR REPLACE TABLE rfm AS
WITH s AS (
    SELECT household_key,
           CAST(CEIL(5 * CUME_DIST() OVER (ORDER BY recency_days DESC)) AS INTEGER)
AS r,
           CAST(CEIL(5 * CUME_DIST() OVER (ORDER BY baskets_per_week)) AS INTEGER) AS
f,
           CAST(CEIL(5 * CUME_DIST() OVER (ORDER BY spend_per_week)) AS INTEGER) AS m
FROM hh_features
)
SELECT household_key, r, f, m,
CASE WHEN r >= 4 AND (f + m) / 2.0 >= 4 THEN 'Champions'
      WHEN r >= 3 AND (f + m) / 2.0 >= 3 THEN 'Loyal'
      WHEN r >= 4 THEN 'Recent low-value'
      WHEN r <= 2 AND (f + m) / 2.0 >= 3 THEN 'At risk'
      WHEN r <= 2 AND (f + m) / 2.0 < 2 THEN 'Lost'
      ELSE 'Needs attention' END AS segment
FROM s
""");
q("SELECT r, COUNT(*) AS households FROM rfm GROUP BY r ORDER BY r")
```

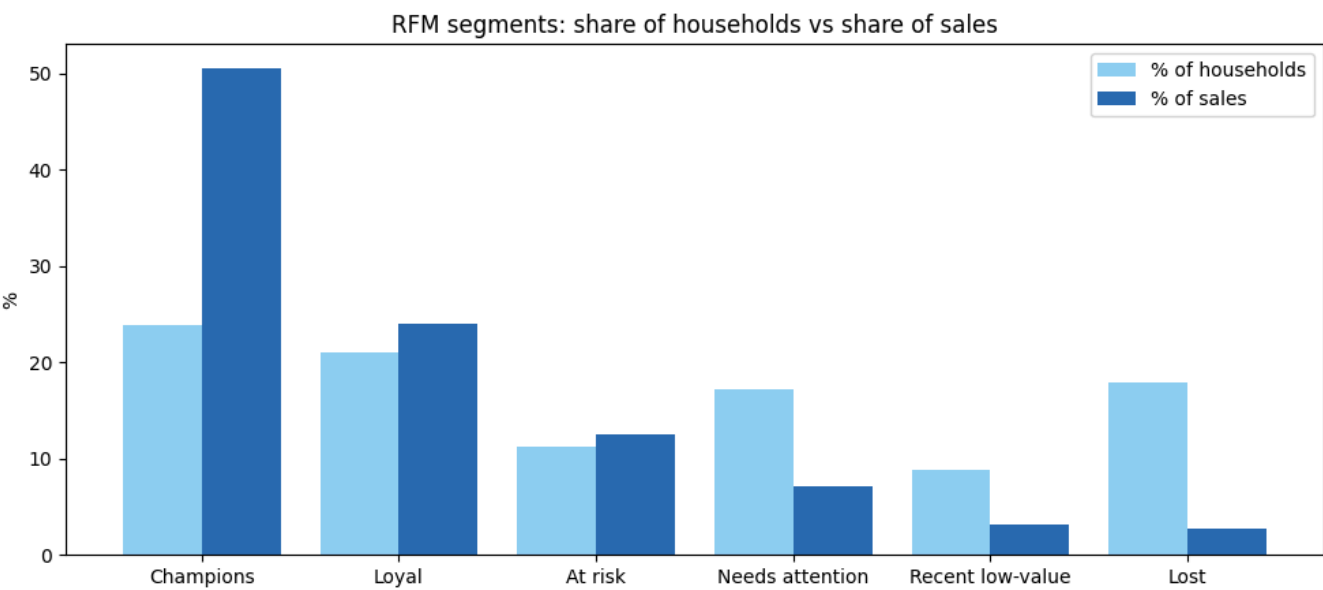
Out

r	households
1	490
2	495
3	439
4	463
5	606

Segments: households vs sales

Share of households against share of sales for each RFM segment.

```
In [5]
seg = q("""
SELECT r.segment,
       COUNT(*) AS households,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS households_pct,
       ROUND(100 * SUM(h.spend) / SUM(SUM(h.spend)) OVER (), 1) AS sales_pct,
       ROUND(AVG(h.spend_per_week), 2) AS avg_spend_per_week,
...
FROM rfm r
JOIN hh_features h ON h.household_key = r.household_key
GROUP BY r.segment
ORDER BY sales_pct DESC
""")
...
```



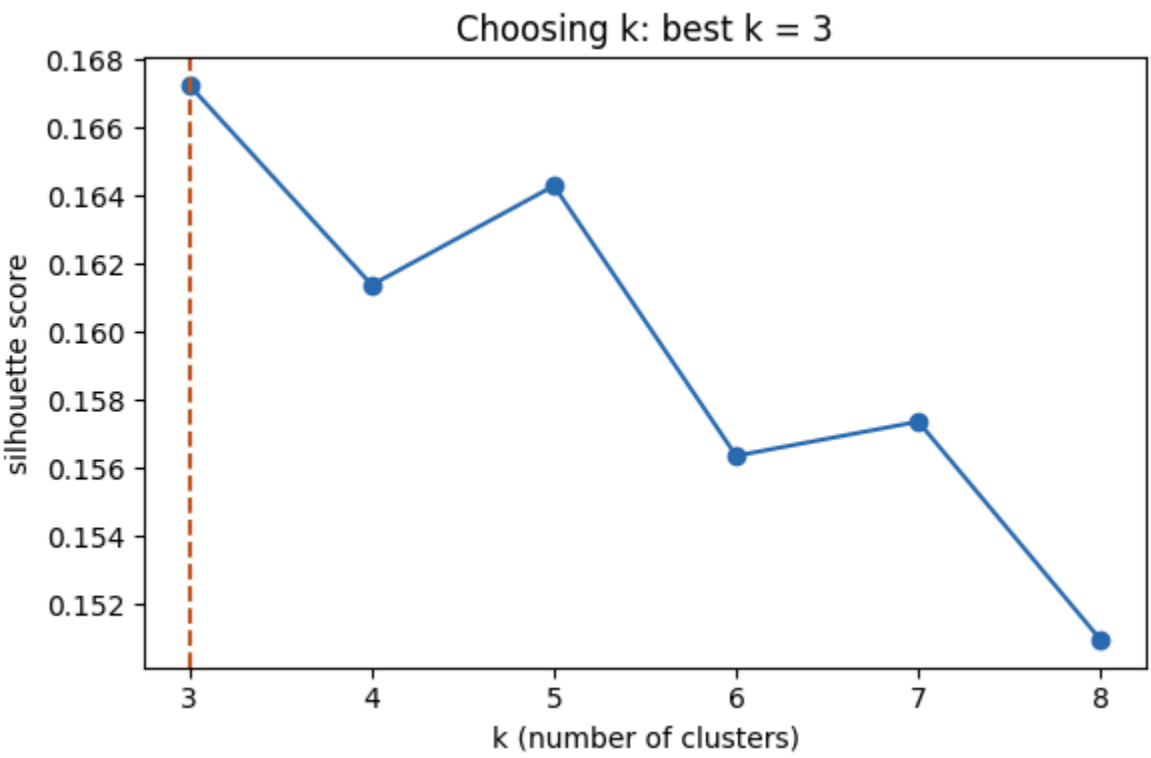
Out

segment	households	households_pct	sales_pct	avg_spend_per_week
Champions	594	23.8	50.5	68.20
Loyal	523	21.0	24.0	37.00
At risk	280	11.2	12.5	36.48
Needs attention	428	17.2	7.1	13.74
Recent low-value	222	8.9	3.1	11.79
Lost	446	17.9	2.7	5.12

Cross-checking RFM with k-means

Seven log-scaled behaviour features, number of clusters chosen by silhouette score.

```
In [6]
feats = ['recency_days', 'baskets_per_week', 'spend_per_week', 'avg_basket_value',
'departments', 'private_share', 'promo_share']
X = q("SELECT * FROM hh_features").set_index('household_key')
Z = StandardScaler().fit_transform(np.log1p(X[feats]))
sil = {k: silhouette_score(Z, KMeans(k, n_init=10, random_state=42).fit_predict(Z))
for k in range(3, 9)}
best_k = max(sil, key=sil.get)
...
```



Out

	silhouette
3	0.167
4	0.161
5	0.164
6	0.156
7	0.157
8	0.151

One lapse threshold does not fit everyone

Share of visit gaps longer than 23 days, by baseline shopping frequency tier.

```
In [4]
q("""
WITH g AS (
    SELECT v.household_key, LEAD(v.day) OVER (PARTITION BY v.household_key ORDER BY v.day) -
v.day AS gap
    FROM (SELECT DISTINCT household_key, day FROM sales) v
)
SELECT b.frequency_tier, COUNT(*) AS gaps, MEDIAN(g.gap) AS median_gap_days,
    ROUND(100.0 * SUM(CASE WHEN g.gap > 23 THEN 1 ELSE 0 END) / COUNT(*), 1) AS
gaps_over_23_days_pct
FROM g
JOIN baseline b ON b.household_key = g.household_key
WHERE g.gap IS NOT NULL
GROUP BY 1
ORDER BY 1
""")
```

Out

frequency_tier	gaps	median_gap_days	gaps_over_23_days_pct
1: light	28049	7.0	16.3
2: medium	53095	5.0	5.6
3: heavy	108698	3.0	1.1

A lapse threshold per household

Lapse = a gap longer than 3 of the household's own usual cycles, floor 23 days, cap 91 days.

In [5]

```
lapse_sql = lambda k: f"""
...
g AS (
    SELECT v.household_key, v.day, LEAD(v.day) OVER (PARTITION BY v.household_key ORDER BY
v.day) AS next_day,
        LEAST(GREATEST(23, {k} * b.cycle_days), 91) AS thr
    FROM (SELECT DISTINCT household_key, day FROM sales) v
    JOIN baseline b ON b.household_key = v.household_key
),
...
k AS (
    SELECT g.household_key, MIN(g.day) AS lapse_day
    FROM g
    JOIN lb ON lb.household_key = g.household_key
    CROSS JOIN p
    WHERE g.day >= lb.last_base
        AND COALESCE(g.next_day, 999999) - g.day > g.thr
        AND g.day + g.thr <= p.end_day
    GROUP BY g.household_key
)
...
surv = q(f"SELECT l.*, b.* EXCLUDE (household_key) FROM ({lapse_sql(3)}) l JOIN baseline b
ON b.household_key = l.household_key").assign(weeks=lambda d: d.days / 7)
surv.groupby('frequency_tier').agg(households=('lapsed', 'size'), lapsed_pct=('lapsed',
lambda s: round(100 * s.mean(), 1)),
                                returned_after_lapse_pct=('returned', lambda s:
round(100 * s[surv.loc[s.index, 'lapsed']].mean(), 1)))
```

Out

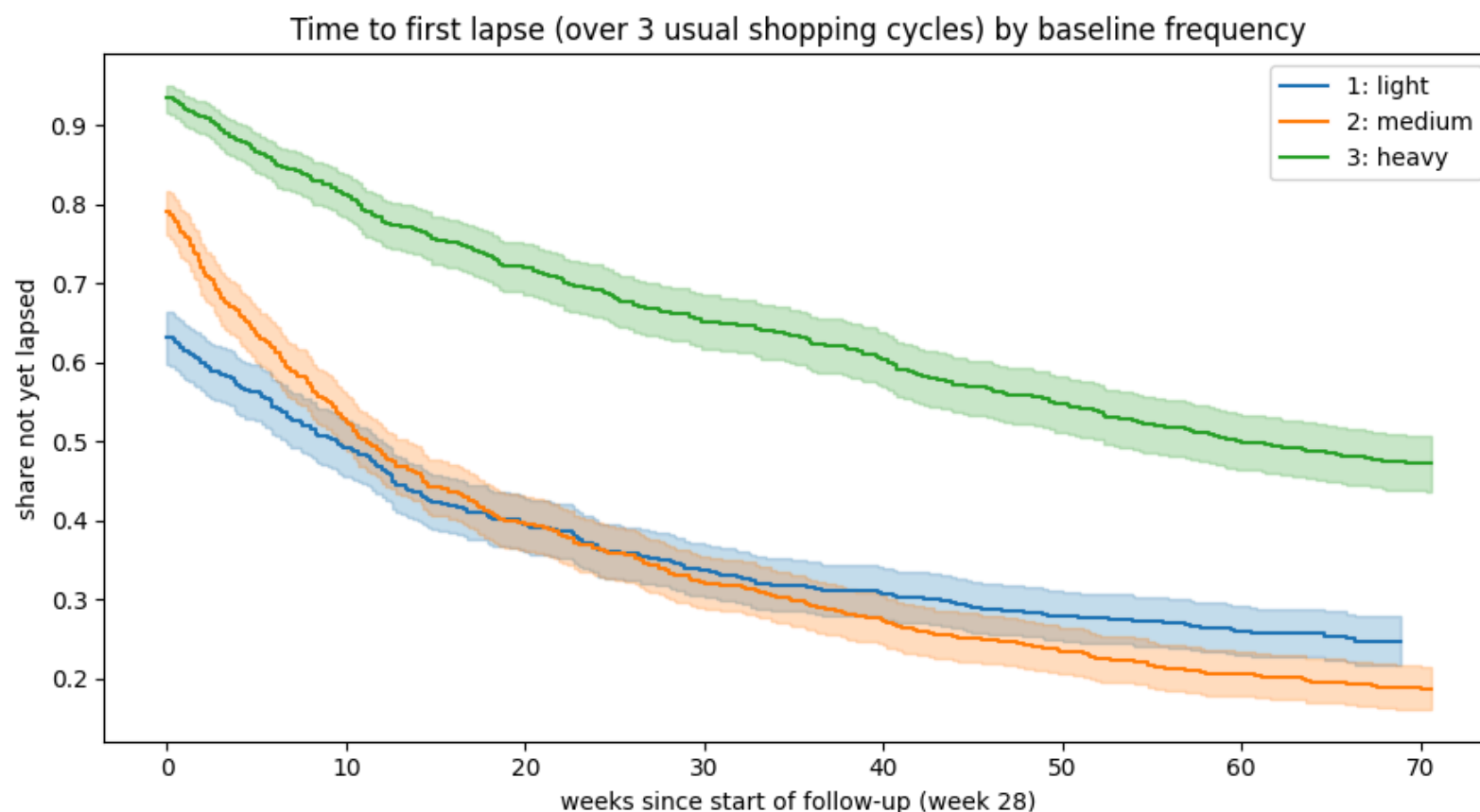
frequency_tier	households	lapsed_pct	returned_after_lapse_pct
1: light	777	74.8	97.1
2: medium	777	81.3	98.9
3: heavy	776	52.8	98.8

Kaplan-Meier by frequency tier

Share of households not yet lapsed over time, with a log-rank test across tiers.

In [6]

```
fig, ax = plt.subplots(figsize=(9, 5))
for tier, d in surv.groupby('frequency_tier'):
    KaplanMeierFitter().fit(d.weeks, d.lapsed,
label=tier).plot_survival_function(ax=ax)
...
lr = multivariate_logrank_test(surv.weeks, surv.frequency_tier, surv.lapsed)
print(f"Log-rank test: chi2 = {lr.test_statistic:.1f}, p = {lr.p_value:.3g}")
...
```



Log-rank test: chi2 = 217.0, p = 7.72e-48

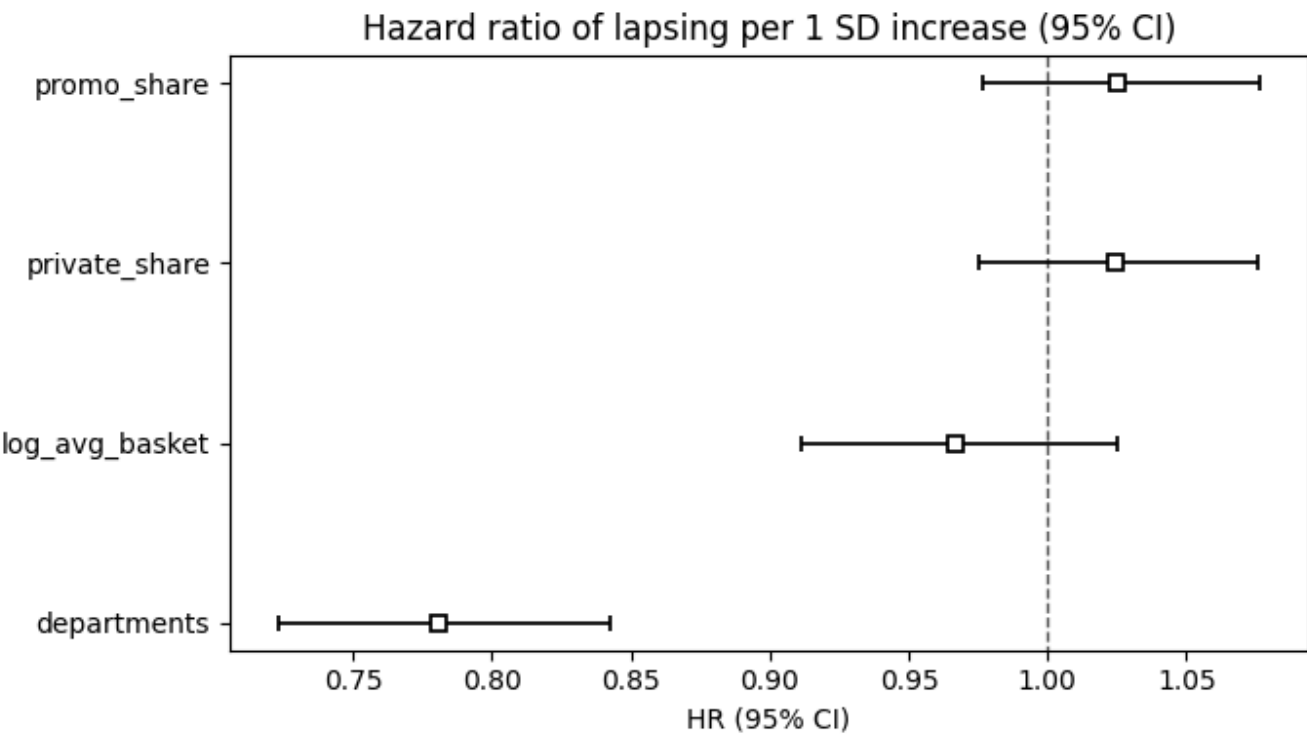
Out

	median_weeks_to_lapse	not_lapsed_at_26w_pct	not_lapsed_at_52w_pct
1: light	9.428571	35.8	27.5
2: medium	11.142857	35.5	22.5
3: heavy	59.714286	67.3	53.7

Cox model: what predicts lapsing?

Stratified by frequency tier, covariates standardised, hazard ratio per 1 SD.

```
In [7]
cox_df = surv[['weeks', 'lapsed', 'frequency_tier']].assign(
    log_avg_basket=np.log1p(surv.avg_basket), departments=surv.departments,
    private_share=surv.private_share, promo_share=surv.promo_share)
covs = cox_df.columns[3:]
cox_df[covs] = (cox_df[covs] - cox_df[covs].mean()) / cox_df[covs].std()
cph = CoxPHFitter().fit(cox_df, duration_col='weeks', event_col='lapsed', strata=
['frequency_tier'])
...
```



Concordance = 0.579

Out

covariate	exp(coef)	exp(coef) lower 95%	exp(coef) upper 95%	p
log_avg_basket	0.966	0.911	1.025	0.258
departments	0.781	0.723	0.842	0.000
private_share	1.024	0.976	1.076	0.334
promo_share	1.025	0.976	1.076	0.318

Market basket analysis in pure SQL

A self-join lists every commodity pair in every basket once. Support, confidence, lift and leverage follow.

In [5]

```
con.execute(f"""
...
CREATE OR REPLACE TABLE pairs AS
WITH n AS (SELECT COUNT(DISTINCT basket_id) AS n FROM bc),
p AS (
  SELECT a.commodity AS item_a, b.commodity AS item_b, COUNT(*) AS n_ab
  FROM bc a
  JOIN bc b ON b.basket_id = a.basket_id AND a.commodity < b.commodity
  WHERE a.commodity IN (SELECT commodity FROM items) AND b.commodity IN (SELECT commodity
FROM items)
  GROUP BY 1, 2
)
SELECT p.item_a, da.department AS dept_a, p.item_b, db.department AS dept_b, p.n_ab,
       1.0 * p.n_ab / n.n AS support,
       1.0 * p.n_ab / ia.n_baskets AS conf_a_to_b,
       1.0 * p.n_ab / ib.n_baskets AS conf_b_to_a,
       1.0 * p.n_ab * n.n / (1.0 * ia.n_baskets * ib.n_baskets) AS lift,
       1.0 * p.n_ab / n.n - (1.0 * ia.n_baskets / n.n) * (1.0 * ib.n_baskets / n.n) AS
leverage
FROM p
JOIN items ia ON ia.commodity = p.item_a
JOIN items ib ON ib.commodity = p.item_b
JOIN item_dept da ON da.commodity = p.item_a
JOIN item_dept db ON db.commodity = p.item_b
CROSS JOIN n
WHERE p.n_ab >= {MIN_PAIR} * n.n;
""");
q("SELECT (SELECT COUNT(*) FROM items) AS frequent_commodities, COUNT(*) AS pairs_kept,
SUM(CASE WHEN lift > 1 THEN 1 ELSE 0 END) AS pairs_with_lift_above_1 FROM pairs")
```

Out

frequent_commodities	155
pairs_kept	6499
pairs_with_lift_above_1	6428.0

Strongest product links

Top commodity pairs ranked by lift.

```
In [6]
show = lambda order, where='TRUE': q(f"""
SELECT item_a, dept_a, item_b, dept_b, n_ab AS baskets, ROUND(100 * support, 2) AS
support_pct,
        ROUND(100 * conf_a_to_b, 1) AS conf_a_to_b_pct, ROUND(100 * conf_b_to_a, 1)
AS conf_b_to_a_pct,
        ROUND(lift, 2) AS lift, ROUND(100 * leverage, 3) AS leverage_pct
FROM pairs
WHERE {where}
ORDER BY {order} DESC
LIMIT 15
""")
show('lift')
```

Out

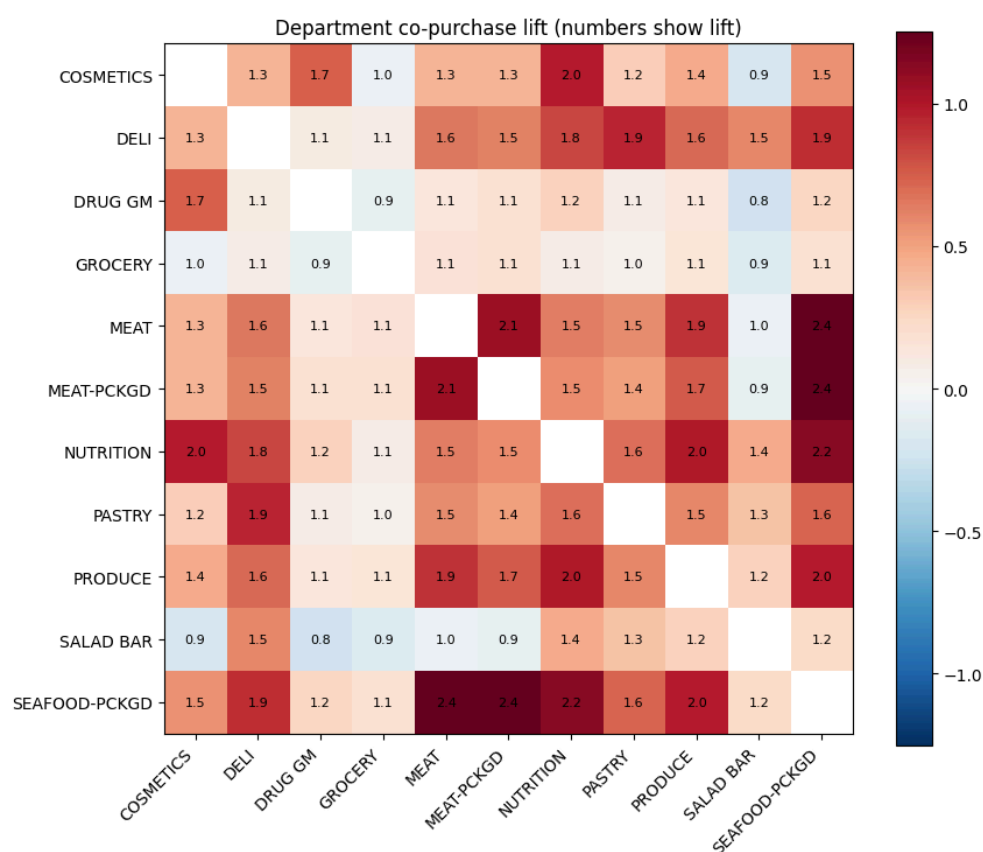
item_a	item_b	baskets	support_pct	conf_a_to_b_pct	lift
CAT FOOD	CAT LITTER	1185	0.51	19.4	16.40
BABY FOODS	INFANT FORMULA	669	0.29	17.1	16.40
BABY HBC	DIAPERS & DISPOSABLES	1005	0.44	27.4	15.03
LAUNDRY ADDITIVES	LAUNDRY DETERGENTS	1731	0.75	42.4	12.55
BROOMS AND MOPS	HOUSEHOLD CLEANG NEEDS	950	0.41	34.2	11.56
BLEACH	LAUNDRY ADDITIVES	646	0.28	19.9	11.23
DEODORANTS	SHAVING CARE PRODUCTS	563	0.24	14.5	10.99
BLEACH	LAUNDRY DETERGENTS	1171	0.51	36.0	10.67
DOG FOODS	PET CARE SUPPLIES	697	0.30	10.5	10.42
DRY NOODLES/PASTA	PASTA SAUCE	5940	2.57	45.6	9.70

Department co-purchase map

The same lift logic at department level. Red means bought together more often than chance.

In [9]

```
dL = q("""
WITH bd AS (SELECT DISTINCT basket_id, department FROM sales),
n AS (SELECT COUNT(DISTINCT basket_id) AS n FROM bd),
d AS (SELECT department, COUNT(*) AS n_d FROM bd GROUP BY department HAVING COUNT(*) >=
0.02 * (SELECT n FROM n)),
p AS (
SELECT a.department AS dept_a, b.department AS dept_b, COUNT(*) AS n_ab
FROM bd a
JOIN bd b ON b.basket_id = a.basket_id AND a.department < b.department
WHERE a.department IN (SELECT department FROM d) AND b.department IN (SELECT department
FROM d)
GROUP BY 1, 2
)
SELECT p.dept_a, p.dept_b, 1.0 * p.n_ab * n.n / (1.0 * da.n_d * db.n_d) AS lift
FROM p
JOIN d da ON da.department = p.dept_a
JOIN d db ON db.department = p.dept_b
CROSS JOIN n
""")
...
```



Finding clean campaign windows

A household counts as treated only if no other campaign overlaps its pre-period or campaign weeks.

In [3]

```
clean = lambda w: f"COUNT(*) FILTER (WHERE NOT EXISTS (SELECT 1 FROM hh_camp o\nWHERE o.household_key = a.household_key AND o.campaign <> a.campaign AND\no.start_day <= a.end_day AND o.end_day >= a.start_day - {7 * w})) AS\nclean_pre_{w}w"\ndesign = q(f"SELECT campaign, type, start_week, end_week, COUNT(*) AS targeted, {'\n'.join(clean(w) for w in (4, 6, 8))} FROM hh_camp a GROUP BY ALL ORDER BY\nstart_week")\npd.concat([design,\ndesign.sum(numeric_only=True).to_frame('total').T.assign(campaign='', type='',\nstart_week='', end_week='')])
```

Out

campaign	type	targeted	clean_pre_4w	clean_pre_6w	clean_pre_8w
26	TypeA	332	310	310	310
27	TypeC	12	1	1	1
28	TypeB	17	2	2	2
29	TypeB	118	77	77	77
30	TypeA	361	260	260	260
...					
8	TypeA	1076	641	610	485
13	TypeA	1077	602	524	129
18	TypeA	1133	510	165	165
...					
total		7208	2673	2210	1682

Propensity score matching

Nearest neighbours on the logit within each campaign, inside a caliper, with two control groups.

```
In [6]
def match(df, by=('campaign',)):
    Z = np.hstack([StandardScaler().fit_transform(df[X]),
pd.get_dummies(df.campaign).values])
    p = LogisticRegression(max_iter=5000).fit(Z, df.treated).predict_proba(Z)[:,
1].clip(1e-6, 1 - 1e-6)
    df = df.assign(lps=np.log(p / (1 - p)))
    cal = CALIPER * df.lps.std()
    pairs = []
    for _, g in df.groupby(list(by)):
        t, c = g[g.treated], g[~g.treated]
        if len(t) and len(c):
            d, i = NearestNeighbors(n_neighbors=min(K,
len(c))).fit(c[['lps']]).kneighbors(t[['lps']])
            pairs.append(pd.DataFrame({'t': np.repeat(t.index, d.shape[1]), 'c':
c.index.values[i.ravel()], 'd': d.ravel()}))
            pr = pd.concat(pairs).query('d <= @cal').assign(w=lambda x: 1 /
x.groupby('t').t.transform('size'))
            w = pd.concat([pd.Series(1.0, index=pr.t.unique()), pr.groupby('c').w.sum()])
            return df.assign(w=w.groupby(level=0).sum()).assign(matched=lambda x: x.w.notna())
...
groups = {'never': 'never targeted', 'other': 'targeted at another time'}
m = {g: match(u[u.grp.isin(['treated', g])].copy()) for g in groups}
...
```

Out

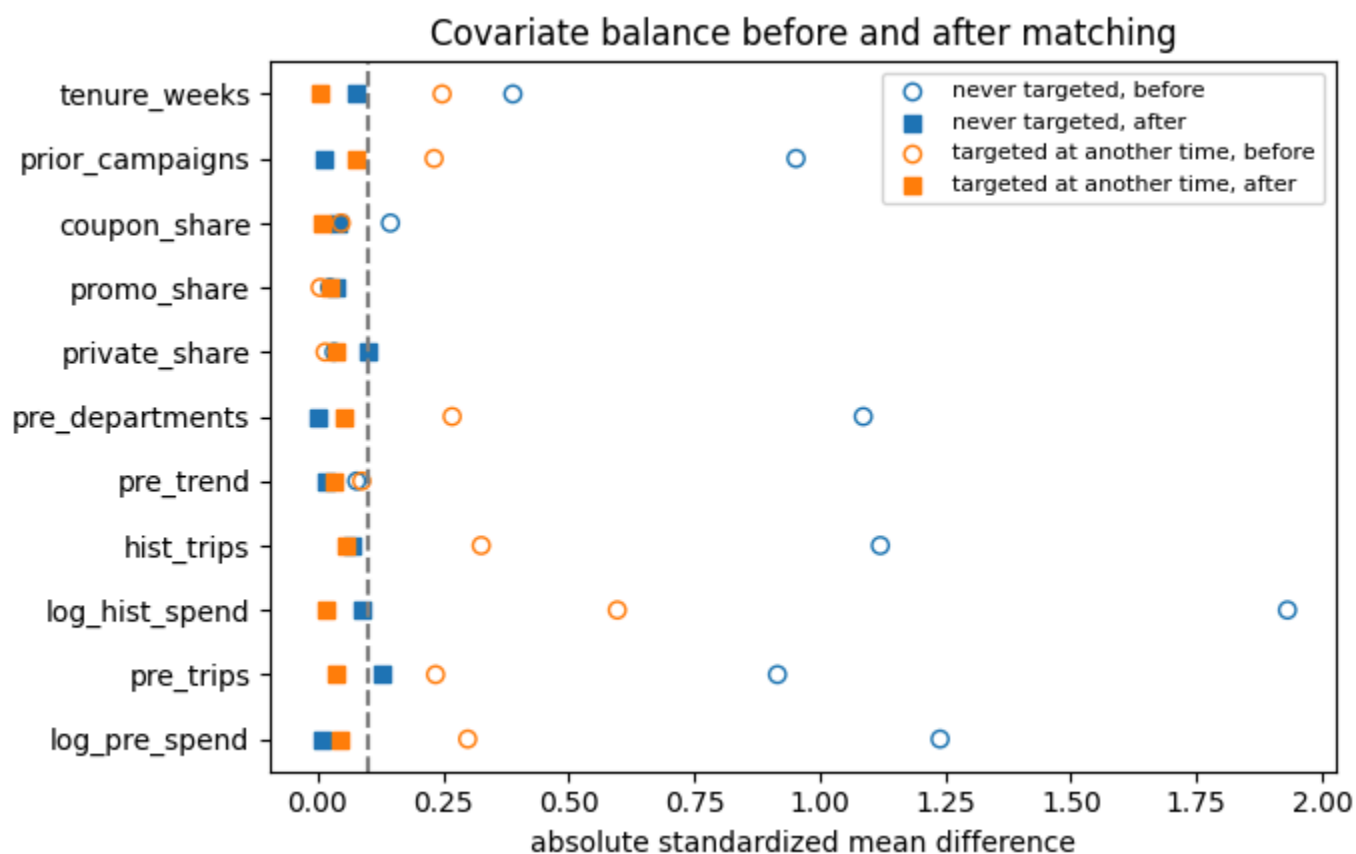
	value
never: treated matched %	58.0
never: controls used	1180.0
never: pre spend, all treated	43.5
never: pre spend, matched treated	43.4
other: treated matched %	99.9
other: controls used	2314.0
other: pre spend, all treated	43.5
other: pre spend, matched treated	43.4

Covariate balance after matching

Standardised mean differences before and after matching. Under 0.1 counts as balanced.

In [8]

```
bal = pd.concat({(g, s): smd(d, m[g]) for g in groups for s, d in (('before',  
m[g].assign(w=1.0)), ('after', m[g][m[g].matched]))}, axis=1).round(3)  
...
```



Difference-in-differences

Household-level regression $dy \sim \text{treated} + C(\text{campaign})$, matching weights, household-clustered SE.

In [9]

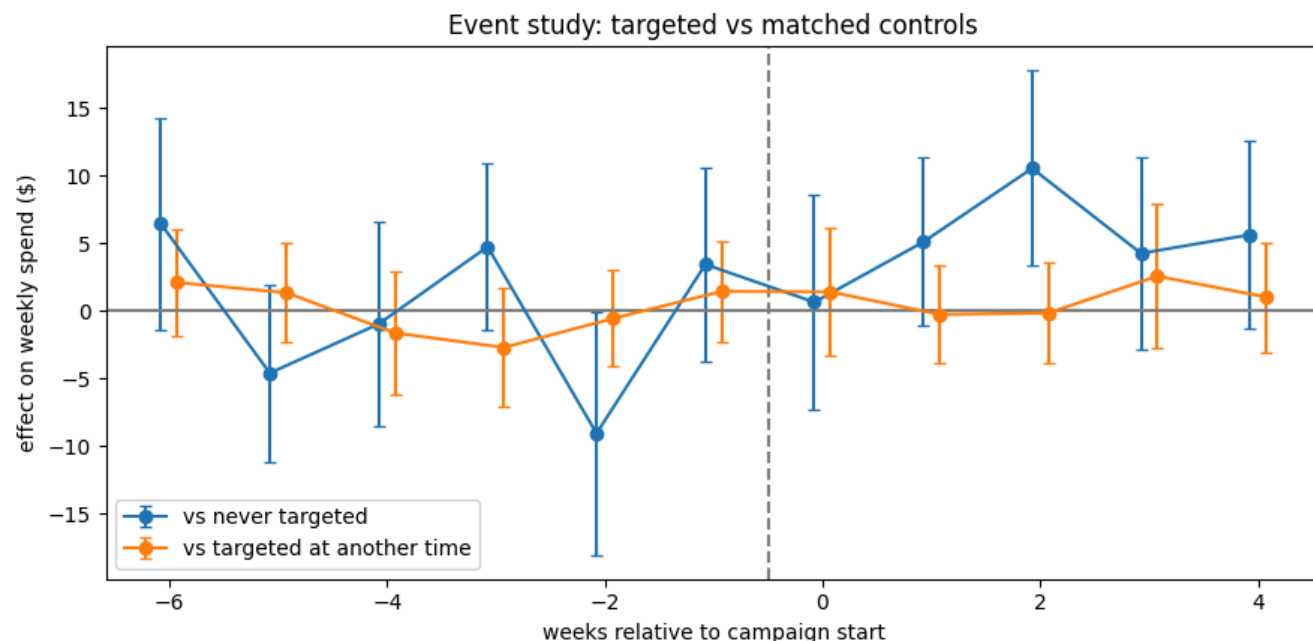
```
def did(df, y, post='during', pre='pre'):
    d = df.dropna(subset=['w', f'{y}_{post}', f'{y}_{pre}']).assign(dy=lambda x:
x[f'{y}_{post}'] - x[f'{y}_{pre}'])
    r = smf.wls('dy ~ treated + C(campaign)', d,
weights=d.w).fit(cov_type='cluster', cov_kws={'groups': d.household_key})
    b, (lo, hi), base = r.params['treated[T.True]'],
r.conf_int().loc['treated[T.True]'], d.loc[d.treated, f'{y}_{pre}'].mean() or
np.nan
    return dict(estimate=b, ci_low=lo, ci_high=hi, p=r.pvalues['treated[T.True]'],
pct_of_pre=100 * b / base,
                treated_pre_mean=base, n_treated=int(d.treated.sum()),
n_controls=int((~d.treated).sum()))
res = []
add = lambda analysis, group, y, **k: res.append(dict(analysis=analysis,
control_group=group, outcome=y, **k))
naive = u[u.grp.isin(['treated', 'never'])].assign(w=1.0, spend_zero=0.0)
add('1. naive: campaign weeks only', 'never', 'spend', **did(naive, 'spend',
'during', 'zero'))
add('2. DiD, unmatched', 'never', 'spend', **did(naive, 'spend'))
for g in groups:
    add('3. DiD, matched', g, 'spend', **did(m[g], 'spend'))
show = lambda r: pd.DataFrame(r).round({'estimate': 2, 'ci_low': 2, 'ci_high': 2,
'p': 4, 'pct_of_pre': 1, 'treated_pre_mean': 2})
show(res)
```

Out

analysis	control_group	estimate	ci_low	ci_high	p
1. naive: campaign weeks only	never	31.40	28.94	33.85	0.0000
2. DiD, unmatched	never	1.83	0.63	3.02	0.0027
3. DiD, matched	never	5.61	1.80	9.43	0.0039
3. DiD, matched	other	1.03	-0.99	3.05	0.3170

Event study and placebo test

Were trends parallel before the campaign? A fake start date should show no effect.



In [12]

```
pw = q("""
SELECT campaign, household_key,
       AVG(spend) FILTER (WHERE event_week < -{PRE // 2}) AS spend_early,
       AVG(spend) FILTER (WHERE event_week BETWEEN -{PRE // 2} AND -1) AS spend_late
FROM panel WHERE period = 'pre' GROUP BY ALL
""")
n0 = len(res)
for g in groups:
    add('5. placebo: fake start mid pre-period', g, 'spend', **did(m[g].merge(pw, on=
['campaign', 'household_key']), 'spend', 'late', 'early'))
...
```

Out

analysis	control_group	outcome	estimate	p
5. placebo: fake start mid pre-period	never	spend	-0.58	0.8351
6. after window	never	spend	7.06	0.0008
6. after window	never	coupon_spend	0.96	0.1214
5. placebo: fake start mid pre-period	other	spend	-1.21	0.4604
6. after window	other	spend	-2.10	0.0930
6. after window	other	coupon_spend	-0.45	0.4425

Export for Power BI, then validate

10 star-schema tables written to Parquet. Every check must pass before the model is built.

In [3]

```
checks = {
    'sales match notebook 03': "SELECT ABS(SUM(sales_value) - 6814347) < 1 FROM
pbi_fact_sales",
    'households match notebook 03': "SELECT COUNT(DISTINCT household_key) = 2493
FROM pbi_fact_sales",
    'baskets match notebook 03': "SELECT COUNT(DISTINCT basket_id) = 230892 FROM
pbi_fact_sales",
    ...
pd.DataFrame({'passed': {c: con.sql(s).fetchone()[0] for c, s in checks.items()}})
```

Out

	passed
sales match notebook 03	True
households match notebook 03	True
baskets match notebook 03	True
dim_product key unique	True
dim_household key unique	True
dim_week key unique	True
dim_campaign key unique	True
day_of_week key unique	True
fact_sales -> dim_product	True
fact_sales -> dim_household	True
fact_sales -> dim_week	True
fact_sales -> day_of_week	True
bridge_household_campaign -> dim_household	True
bridge_household_campaign -> dim_campaign	True